Foundations of Computer Programming: Computational Thinking

Learning objectives:

- Developing a Comprehensive Understanding of Core Computational Thinking Concepts
- Understand Problem Decomposition
- Understand Pattern Recognition
- Understand Abstraction
- Understand Algorithm Thinking

Lesson exercise: Week 2

In this exercise, you will work on understanding the core computational thinking concepts and how they can be applied to problem-solving.

1. What is computational thinking?

Computers can be leveraged to assist in solving problems. However, before a problem can be addressed, it is crucial to comprehend the problem itself and explore potential solutions. Computational thinking enables us to do just that. This approach allows us to tackle complex problems, understand their core essence, and develop viable solutions. We can then present these solutions in a manner that can be understood by computers, humans, or both.

2. What are the four cornerstones of computational thinking?

The four key techniques of computational thinking are:

Decomposition - breaking down a complex problem or system into smaller, more manageable parts

Pattern recognition – looking for similarities among and within problems

Abstraction – focusing on the important information only, ignoring irrelevant detail

Algorithms - developing a step-by-step solution to the problem, or the rules to follow to solve the problem

Each cornerstone is equally vital to the overall framework. They are akin to the legs supporting a table - if one is absent, the entire structure is prone to collapse. Properly employing all four techniques will significantly aid in computer programming endeavors.

3. How to apply computational thinking to problem solving?

To apply computational thinking to problem-solving, one can follow these general steps:

Decompose the problem into smaller, more manageable parts.

Identify patterns in the problem or its solution that can be generalised.

Abstract away unnecessary details to focus on the core issues.

Develop an algorithmic solution by breaking the problem down into a series of precise steps.

By consistently applying these four techniques, complex problems can be systematically broken down and solutions can be effectively designed, even for problems that initially appear daunting.

4. Computational thinking test questions:

I. Which of the following is NOT a core technique of computational thinking?

   a. Debugging

   b. Decomposition

   c. Pattern recognition

   d. Abstraction

Answer: a) Debugging. The four core techniques are decomposition, pattern recognition, abstraction, and algorithm design.

II. The process of breaking down a complex problem into smaller, more manageable parts is called:

   a. Abstraction

   b. Decomposition

   c. Pattern recognition

   d. Algorithm design

Answer: b) Decomposition

III. Focusing on the important information and ignoring irrelevant details is an example of:

a. Abstraction

b. Decomposition

c. Pattern recognition

d. Algorithm design

Answer: a) Abstraction

IV. The step-by-step solution or set of rules to solve a problem is known as:

a. Abstraction

b. Decomposition

c. Pattern recognition

d. Algorithm design

Answer: d) Algorithm design

5. Try more: https://www.bbc.co.uk/bitesize/guides/zp92mp3/test

6. For the following tasks, choose a group of 3-5 students to work on each exercise.

- Choose a different real-world activity
- Share your findings with the class.

I. Imagine you are planning a birthday party. List all the tasks required to organise the event. Break the event planning process into smaller steps.

Example:

When planning a birthday party, there are several tasks to consider. First, choose a date and time, create a guest list and send invitations. Next, decide on a theme and decorations that align with it. Plan the menu, including any dietary requirements, and order a cake. Arrange for entertainment, such as music or activities. Ensure you have the

necessary supplies, like tableware, napkins, and party favours. Set up the venue and arrange the decorations. During the event, greet guests, serve food and drinks, and facilitate the planned activities. After the party, clean up the venue, pack up any remaining items, and express gratitude to those who attended.

II. Examine the following numerical sequences. Identify any discernible patterns, then predict the next number in each sequence.

a) 2, 4, 6, 8, ...

Pattern: The numbers are increasing by 2.

Next number: 10.

b) 5, 10, 20, 40, ...

Pattern: The numbers are doubling each time.

Next number: 80.

c) 1, 4, 9, 16, ...

Pattern: These are square numbers (1^2, 2^2, 3^2, 4^2).

Next number: 25 (5^2).

III. Imagine you are giving directions to a friend to get from their house to your favourite restaurant. However, instead of describing every single landmark or street corner, simplify your directions by only including the important parts.

Example:

“Start by heading out from your house and turn right onto Main Street. Follow that for about 3 blocks, then turn left onto One Avenue. The restaurant will be on your right-hand side, just past the bank. I would recommend looking for the big neon sign - that's how you will know you've arrived. Let me know if you need any clarification on the

directions!"

IV. Write a simple algorithm for making a sandwich. Each step must be clear and easy to follow, as if you're teaching someone who has never made a sandwich before.

An example algorithm:

- Gather all necessary ingredients: two slices of bread, a spread, and any desired toppings.
- Spread butter on one slice.
- Add a layer of cheese on the buttered side.
- Place a slice of ham on top of the cheese.
- Put the second slice of bread on top.
- Cut the sandwich in half and serve.

7. Homework:

You have been asked to organise a school charity event. Use the four computational thinking concepts to solve this problem.

Example

Decompose the problem: Break the event planning process into smaller tasks. (e.g., Booking the venue, organising entertainment, advertising, collecting donations).

Recognise patterns: Identify any patterns from previous events (e.g., time required to plan, common challenges, popular activities).

Abstract the problem: Focus on the essential details (e.g., raising funds, getting people to attend) and ignore unnecessary complexities (e.g., what colour the decorations should be).

Design an algorithm: Write a step-by-step process for planning the event, from start to finish.